
Sparse Attention as a Range Searching Problem: Towards an Inference-Efficient Index for KV Cache

Mohsen Dehghankar

Department of Computer Science
University of Illinois Chicago
mdehgh2@uic.edu

Abolfazl Asudeh

Department of Computer Science
University of Illinois Chicago
asudeh@uic.edu

Abstract

Sparse attention improves LLM inference efficiency by selecting a subset of key-value (KV) entries, but at the cost of potential accuracy degradation. In particular, omitting critical KV entries can induce substantial errors in model outputs. Existing methods typically operate under fixed or adaptive token budgets and provide empirical robustness or partial theoretical guarantees, yet they do not ensure zero false negatives across decoding steps, particularly since the set of relevant tokens is both query- and step-dependent. Our empirical observations confirm that missing even one critical key can lead to sharp error spikes, especially in long-output reasoning tasks where the set of important tokens varies throughout decoding. This observation motivates the need for indexing methods that dynamically adapt to these variations across decoding steps while guaranteeing a full recall of the relevant keys above a certain threshold. We address this challenge by reformulating sparse attention as the computational geometry problem of halfspace range searching. However, existing range searching data structures are not suitable for modern LLM inference due to their computational and implementation overheads. To overcome this, we introduce **Louver**, a novel index structure tailored for efficient KV cache retrieval. **Louver** (i) guarantees zero false negatives with respect to a specified threshold in both theory and practice, (ii) is lightweight to integrate into existing LLM inference pipelines, and (iii) incorporates hardware-aware optimizations for both CPU and GPU executions. Our experiments demonstrate that **Louver** outperforms prior sparse attention methods in both accuracy and runtime, and is faster than highly optimized dense implementations such as FlashAttention. These results highlight that recall guarantees are a critical and overlooked dimension of sparse attention, and open a new direction for building theoretically grounded, efficient KV cache indices. Code is available at <https://github.com/UIC-InDeXLab/Louver>.

1 Introduction

“... the ocean would be less because of that missing drop.” —Mother Teresa

Large Language Models (LLMs) rely on the Key-Value (KV) cache to reuse previously computed key and value vectors for efficient inference. While effective, the KV cache introduces significant memory and computational overhead, as each newly generated token attends to all prior tokens. A key observation is that attention is inherently sparse: for any query token, only a small subset of past tokens, called “active tokens” meaningfully contributes to its representation [10, 6]. This sparsity has motivated a large body of work on accelerating inference via selective attention over the KV cache.

Example 1. As a running example, let the last token in a context window be $t_n = [\text{paper}]$, with the vector x_n encoding the “description” of the general concept [paper]. Suppose the context contains:

[...We attended **NeurIPS**, **KDD**, **SIGIR**, and **VLDB** last year, to present our **papers**]

The highlighted tokens [NeurIPS], [KDD], [SIGIR], and [VLDB] are the active tokens that contribute to refining x_n , shifting it toward concepts such as Computer Science, Machine Learning, Information Retrieval, Data Mining, Data Management, etc, by inducing an update $\tilde{o}_n$ as $x_n \leftarrow x_n + \tilde{o}_n$. Importantly,

1. the number of active tokens is not fixed, as it depends on the context, and
2. each token may contribute a distinct semantic aspects, making their inclusion critical.

Existing KV-cache acceleration techniques often rely on selecting a fixed number k of tokens per query (e.g., top- k retrieval or approximate nearest neighbors) [32, 9, 50, 13, 22]. Conversely, the number of active tokens is inherently *query-dependent* and cannot be determined a priori. As illustrated in Example 1, the number of relevant tokens (e.g., conferences attended) can vary arbitrarily, making any fixed value of k either insufficient or wasteful.

However, even more critically, these methods can introduce **false negatives**, i.e., they may miss tokens that significantly contribute to the attention computation. This issue arises from two sources: (i) the approximate nature of the underlying retrieval algorithms, and (ii) the use of a fixed k , which enforces an artificial cap on the number of retrieved tokens. Missing an active token can lead to incomplete or biased updates of the query embedding. In Example 1, omitting [VLDB] would remove the data management aspect from $\tilde{o}_n$, degrading the semantic fidelity of the model’s representation. As a result, unlike many retrieval settings, *sparse attention is highly sensitive to missing relevant tokens*. This observation fundamentally challenges the prevailing design of *approximate* or top- k retrieval for KV caching and calls for mechanisms that prevent *false negatives*.

Our Perspective. In this paper, we revisit KV-cache sparsification through the lens of *exact* active-token retrieval. We argue that the problem should not be formulated as selecting a fixed-size or as an approximation objective, but rather as identifying all relevant tokens whose contribution exceeds a query-dependent threshold. In particular, our contributions can be summarized as follows:

- We reduce the sparse attention problem to the *range searching problem*. Through this reduction, we show that an exact solution for the range searching problem retrieves all tokens whose similarity with the query exceeds a dynamically determined threshold. This formulation addresses both limitations: it eliminates the need for a fixed budget k and prevents false negatives.
- The existing range-searching data structures are not suitable for LLM inference. We identify a set of index design challenges for KV-cache sparsification.
- Building on the design challenges, we develop an efficient indexing framework, called LOUVER, that supports dynamic, threshold-based retrieval over the KV cache, ensuring exactness while achieving significant computational savings.
- We introduce various system-level optimizations to make Louver deployment-ready to integrate into existing LLM-inference pipelines, develop a lightweight oracle for dynamically identifying the relevance threshold, and incorporate hardware optimizations for CPU and GPU executions.
- We conduct comprehensive experiments across fixed-budget, adaptive-budget, KV-offloading, and dense-attention baselines, showing that LOUVER consistently achieves higher accuracy and lower latency than existing methods, while matching or exceeding dense-attention accuracy and achieving up to **15.3 $\times$ GPU** and **10.3 $\times$ CPU** speedups at 40k context length.

Related Work. Prior work on sparse attention for long-context inference spans eviction-based methods, which discard tokens permanently [45, 29, 52], and retrieval-based methods, which dynamically retrieve relevant KV entries using ANN- or hashing-based indexing [32, 9, 13]. Existing approaches typically rely on approximate retrieval or fixed/adaptive token budgets [31, 7, 49], without guaranteeing retrieval of all keys whose attention contribution exceeds the relevance threshold. In contrast, our work formulates sparse attention as an exact range-search problem, enabling dynamically updated indexing with adaptive per-query thresholds. Our approach is complementary to orthogonal acceleration techniques such as FlashAttention [12] and vLLM [26]. Please refer to Appendix B for the detailed discussion of the related work.

2 Preliminaries

Attention. Let $X \in \mathbb{R}^{n \times d}$ denote a sequence of n token representations, where d is the model’s embedding dimension. In *multi-head attention* (MHA), H attention heads operate in parallel, each with its own learned projection matrices, where $d_h = d/H$ is the *head dimension*. Within each

head, X is projected into queries, keys, and values as $Q = XW_Q$, $K = XW_K$, $V = XW_V$ with $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_h}$ and $Q, K, V \in \mathbb{R}^{n \times d_h}$.

For query $q_i \in \mathbb{R}^{d_h}$ and key $k_j \in \mathbb{R}^{d_h}$, the *attention score* is $s_{ij} = \langle q_i, k_j \rangle / \sqrt{d_h}$. Scores are normalized via softmax to obtain *attention weights*: $a_{tj} = \text{softmax}_j(s_{t1}, \dots, s_{t,t-1})$. The per-head output $o_i = \sum_{j \leq i} a_{ij} v_j \in \mathbb{R}^{d_h}$ is computed for each head, concatenated across all H heads, and projected back to $\mathbb{R}^d$ to give $\tilde{o}_i \in \mathbb{R}^d$, which is then added to the residual stream as $x_i \leftarrow x_i + \tilde{o}_i$ before being passed to the next layer. We focus on a single attention head throughout the paper and use d in place of d_h , slightly abusing notation.

Prefilling and Decoding. Transformer inference proceeds in two phases. In *prefilling*, the prompt $x_{1:n}$ is processed in a single parallel forward pass, computing $K, V \in \mathbb{R}^{n \times d}$ simultaneously, producing the initial set of key and value vectors $\{(k_j, v_j)\}_{j=1}^n$ that serve as context for generation. Since all tokens are available at once, prefilling is parallelizable and computationally straightforward.

In *decoding*, tokens are generated one at a time autoregressively: at step t , the model computes query $q_t \in \mathbb{R}^d$ and attends to all $t - 1$ previously computed keys and values, $o_t = \sum_{j=1}^{t-1} a_{tj} v_j$, where a_{tj} is the attention weight restricted to the $t - 1$ cached keys. The output o_t is added to the residual stream and passed to the next layer; after the full forward pass, the final hidden state is then projected onto the vocabulary to sample the next token.

Unlike prefilling, decoding is inherently sequential, and its cost grows with each step, making it the dominant computational bottleneck. This is especially observed in long-output reasoning tasks, where the model generates thousands of tokens [41, 28]. While our methods extend naturally to prefilling, this paper focuses primarily on the decoding phase.

KV Cache. To avoid recomputing keys and values from scratch at each decoding step, the model stores all past key–value pairs in a *KV cache*: $\mathcal{KV}_t = \{(k_j, v_j)\}_{j=1}^{t-1}$, $k_j, v_j \in \mathbb{R}^d$.

At step t , attention requires computing $t - 1$ inner products $\langle q_t, k_j \rangle$ and a weighted sum over all $t - 1$ cached values, so the cost per step grows linearly with sequence length. Grouped query attention (GQA) [4] further reduces the memory footprint of the KV cache by sharing a single key–value head among a group of G query heads. Standard MHA corresponds to $G = 1$.

Sparse Attention. Sparse attention aims to reduce the cost of attention by computing it over only a subset of the KV cache. Instead of attending to all $t - 1$ past keys at decoding step t , the goal is to identify a subset of *relevant* keys (and their corresponding values) that contribute most to the output. Specifically, given a query q_t , sparse attention retrieves a subset of keys $\mathcal{K}^* \subseteq \{k_1, \dots, k_{t-1}\}$ and computes $o_t = \sum_{k_j \in \mathcal{K}^*} a_{tj} v_j$.

Aligned with prior work [31, 53, 54], given a threshold τ , we define the sparse attention problem as finding the set of relevant keys satisfying $\langle q_t, k_j \rangle \geq \tau$; Formally,

Problem 1. Given a query vector $q_t \in \mathbb{R}^d$ at decoding step t , and a set of key vectors $\mathcal{K} = \{k_j \in \mathbb{R}^d\}_{j=1}^{t-1}$, find the subset $\mathcal{K}^* \subseteq \mathcal{K}$, where $\mathcal{K}^* = \{k \in \mathcal{K} \mid \langle q_t, k \rangle \geq \tau\}$.

2.1 Problem Motivation based on Experimental Observations

Before discussing our approach to solving the problem and our reduction to range searching, in the following, we provide two observations that further motivate (a) formulating sparse attention as Problem 1 and (ii) solving it exactly and not approximately.

Observation 1 (Sensitivity to False Negatives). Omitting relevant context tokens leads to significant degradation in attention quality. While an illustrative example is provided in the Introduction, we further demonstrate this effect through a simple experiment. We consider the prompt: “Given a list of numbers $[x_1, x_2, \dots, x_N]$, compute the sum of all elements. Answer with an integer.” We vary the list size N and simulate sparse attention by dropping KV-cache entries. In particular, we remove (i) one token corresponding to a number in the list (i.e., a relevant token), and (ii) one randomly selected non-relevant token. Figure 1a shows that removing a relevant token results in a significant increase in error compared to removing non-relevant tokens. This confirms that all elements in the list are necessary for correctly generating the answer, and that omitting any of them (i.e., introducing false negatives) substantially degrades performance. We further evaluate fixed- K sparse attention under varying list sizes. As shown in Figure 1b, fixed-budget methods exhibit sharp error increases as N grows. This is expected, since the number of relevant tokens grows with the input size while the attention budget remains fixed. In practice, the number of important tokens is unknown, and dropping them can lead to such error spikes. In contrast, retaining all relevant tokens eliminates this issue. As a result, achieving low error (i.e., avoiding error spikes) requires retaining **all relevant** KV-cache

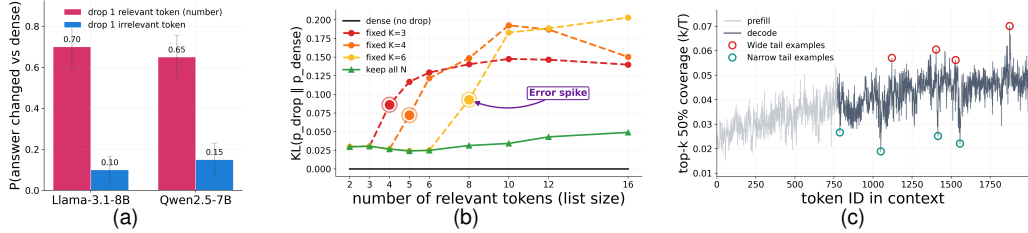


Figure 1: **(a)** Effect of dropping tokens. We remove either (i) one relevant token (a randomly selected number in the list) or (ii) one randomly selected non-relevant token. Dropping a relevant token significantly increases the error. **(b)** Error spike in fixed- K sparse attention as the list size N increases. The green line corresponds to retaining all relevant tokens (no false negatives). **(c)** Top- k coverage across decoding steps, showing significant oscillation and indicating that the number of relevant tokens varies per step (see Observation 2). See Appendices C and D for details.

entries, whose number is not fixed in practice. Prior work has also identified the necessity of adaptive sparsity beyond fixed K [31, 53, 54], typically using coverage-based strategies that select tokens to meet a score threshold. However, developing approximate solutions, these approaches can still omit relevant tokens, as highlighted in the Appendix C.

Observation 2 (Context- and Query-Dependent Score Distribution). Our second observation takes a more computational perspective, focusing on attention scores, i.e., the dot product between query and key vectors. We analyze attention score distributions across different decoding steps in a long reasoning output. For each decoding step, we quantify the tail of the distribution using *top- k coverage*. Larger values indicate that attention is spread over more tokens (wider tails), while smaller values indicate more concentrated attention.¹ As shown in Figure 1c, this metric exhibits strong oscillation across decoding steps. In particular, during reasoning phases such as transitions between thoughts, backtracking, or planning next steps, attention tends to be broader, involving many relevant tokens. In contrast, during more localized computations (e.g., intermediate calculations), attention becomes more concentrated. Explicit examples of these regions are provided in Appendix D.

Observations 1 and 2 indicate that the attention score distribution is inherently *context- and query-dependent*, varying significantly across decoding steps. As a result, it cannot be well-approximated by a fixed pattern or budget throughout inference. Prior work has also observed this dynamic behavior across different attention heads and layers [48, 7, 17]. Together, these findings suggest that attention distributions are dynamic both horizontally (across tokens) and vertically (across layers), motivating the formulation of Problem 1 and the need for exact solutions to it.

3 Reducing Sparse Attention to Halfspace Range Searching

In order to solve Problem 1, we reduce it to the computational geometry problem of *range searching*.

Range searching is defined on a *range space* $(P, \mathcal{R})$ with a finite set of points $P = \{p_1, \dots, p_m\}$, where each point p_i lies in a d -dimensional space $\mathbb{R}^d$ [21]. Associated with P is a family of subsets $\mathcal{R}$, referred to as ranges (e.g., rectangular boxes). Given a range $R \in \mathcal{R}$, the range searching problem retrieves the subset of P that falls inside R . Formally,

Problem 2 (Range Searching [1]). *Given a range space $(P, \mathcal{R})$, preprocesses P into an index so that for any range $R \in \mathcal{R}$, one can retrieve $P \cap R = \{p \in P \mid p \in R\}$, in time sublinear in $|P|$.*

A critical requirement in the range searching problem—and one we enforce throughout this paper—is *100% recall*: all points in $P \cap R$ must be returned, with no false negatives.

Having reviewed the range searching problem, we next provide our reduction:

Reduction. Given an instance of the sparse attention problem (Problem 1) with the query vector $q_t \in \mathbb{R}^d$ and a set of key vectors $\mathcal{K} = \{k_j \in \mathbb{R}^d\}_{j=1}^{t-1}$ in the KV cache at decoding step t , define the following range searching problem (Problem 2):

1. *Points:* For each key $k_j \in \mathcal{K}$, we add a point² to the point set. That is $P \leftarrow \mathcal{K}$.

¹More details in Appendix D.

²From now on, we use terms ‘points’ and ‘keys’ interchangeably.

2. *Ranges*: For the pair (q_t, τ) , we define a *halfspace* R . A halfspace in $\mathbb{R}^d$ is the set of points on one side of a hyperplane. Specifically, we define the halfspace R as

$$R \leftarrow H(q_t, \tau) = \{p \in \mathbb{R}^d \mid \langle q_t, p \rangle \geq \tau\}.$$

3. *Output mapping*: The output of the range searching problem is $P \cap R = \{k \in \mathcal{K} \mid \langle q_t, k \rangle \geq \tau\}$. As a result, $\mathcal{K}^* \leftarrow P \cap R$.

Example 2. To better illustrate the reduction, let us consider the toy example presented in Figure 2. In this example, $d = 2$, i.e., $q_t, k_j \in \mathbb{R}^2$. Each key $k_j \in \mathcal{K}$ is shown as a point in the figure. The query q_t is translated to the green direction, and the threshold τ specifies the dashed line orthogonal to it. The halfspace is the dashed open region bounded by the threshold line. The relevant keys are the blue points inside the halfspace.

Reducing sparse attention to halfspace range searching enables building an index over the keys in the KV cache that supports halfspace range searching queries. A solution to halfspace range searching satisfies **Observation 1**, since by the 1-1 reduction mapping, all relevant keys fall inside the range. It also naturally addresses **Observation 2**: given a threshold τ , the index returns all keys that exceed it, with no fixed assumptions on the number or identity of retrieved keys. As a result, the retrieved set varies per decoding step, adapting dynamically to the query and the current context.

Remark (Comparison to Similarity Search). Traditionally, the sparse attention problem is reduced to a similarity search problem [32, 9, 50, 13, 22], often adapting techniques and indices used for approximate nearest neighbor (ANN) search and maximum inner product search (MIPS). While range searching and similarity search are related problems, they are fundamentally different and suited to different scenarios [2]. The first difference is in the objectives: ANN (resp. MIPS) indices are built to *approximately* find the *top-k* nearest neighbors (resp. max inner product). The second key difference concerns *normalization*. To apply similarity search indexes to KV cache retrieval, one must adopt cosine similarity or angular distance as the retrieval metric [32, 13, 9]. Similarity search indexes based on cosine similarity require all vectors to be normalized [30, 14, 2] to achieve good performance in practice, as they are designed under this assumption. However, keys in transformer attention are not normalized; as we show in Appendix E, their norms vary significantly. Enforcing normalization to fit these indexes effectively alters the key representations that the model was trained with, thereby influencing attention scores and introducing additional errors at inference time. Finally, the third key difference concerns the query-key distribution shift. Similarity search assumes that queries and points come from the same distribution, or at least the same nature. In transformer attention, however, queries and keys are projections of the same token representations through *different* weight matrices, and they live in fundamentally different regimes. This distribution shift is already noted in prior work [32] and remains a challenge for methods that directly adapt similarity search indexes to KV cache retrieval. Halfspace range searching, by contrast, makes no such assumption: the query q_t and the key points $\mathcal{K}$ are treated as geometrically distinct objects.

Index Design Challenges. Classic data structures for range searching are not suitable for LLM inference due to limitations, such as high computational and update overhead. In Appendix F, we identify the three challenges that led our index design choices: (Challenge 1) high pruning power with full recall, (Challenge 2) dynamic and query-adaptive, and (Challenge 3) efficiency and integration.

4 Louver: An Inference-Efficient Index for KV Cache

In this section, we present our index, LOUVER, for KV cache retrieval. We first describe its main components, including index construction, query processing, and dynamic updates. We then analyze the index and show how it addresses the three challenges outlined above. Finally, we discuss the system-level components and implementation details. An overview of

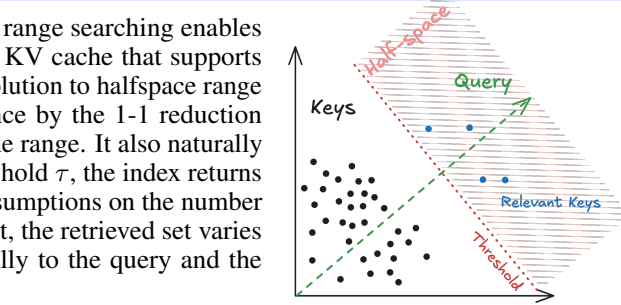


Figure 2: Illustrating the reduction of sparse attention to halfspace range searching (Example 2).

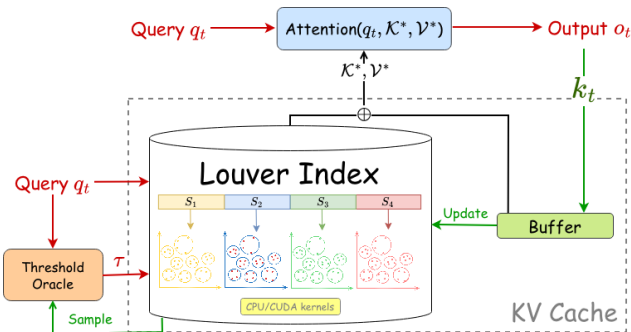


Figure 3: Louver index architecture.

the system architecture is illustrated in Figure 3.

Overview. LOUVER partitions the d -dimensional key space into S contiguous subspaces of approximately equal width and builds an independent index for each. Within each subspace, keys are grouped into clusters of fixed size $r = O(1)$ using a balanced PCA tree, and each cluster is enclosed by a bounding ball (center + radius). The fixed group size enables efficient hardware- and kernel-level optimizations. At query time, given a threshold τ , LOUVER tests each ball against the query halfspace in each subspace. Clusters whose balls do not intersect the halfspace are pruned; only keys belonging to clusters that survive in all S subspaces proceed to an exact dot-product check against τ . This two-phase filter guarantees zero false negatives while avoiding dot-product computation on the large majority of keys.

Keys generated during decoding are held in a small buffer $\mathcal{B}$ and attended densely at every step. Once $|\mathcal{B}|$ reaches B , the buffered keys are clustered into B/r new groups and appended to the index. On GPU, this update runs on a side stream concurrently with attention; on CPU the cost amortizes to $O(1)$ per decoding step.

4.1 Index Construction

Given the KV cache $\mathcal{K} = \{k_1, \dots, k_n\} \subset \mathbb{R}^d$ after prefilling, LOUVER builds a two-level index that decomposes the key space into S subspaces and, within each subspace, groups keys into bounding balls of fixed size r .

Subspace decomposition. We partition the d -dimensional space into S consecutive subspaces of equal size. Each subspace corresponds to a contiguous block of coordinates. For each subspace $s \in \{1, \dots, S\}$, let $\pi_s : \mathbb{R}^d \rightarrow \mathbb{R}^{d_s}$ denote the projection onto that subspace, where $d_s = d/S$. The index for each subspace is built independently, enabling parallel construction and sublinear query cost per subspace. This design also provides effective pruning within each subspace, as discussed in Appendix G.

Grouping via balanced PCA trees. Within each subspace s , we partition the n projected keys $\{\pi_s(k_j)\}_{j=1}^n$ into $K = \lceil n/r \rceil$ groups of fixed size r , following the grouping definition in Appendix G. We construct this partition using a *balanced PCA tree*: starting from all n keys, the tree recursively bisects the current point set by projecting onto the coordinate axis of maximum variance and splitting the sorted sequence at the median, distributing points equally to left and right subtrees. The recursion terminates when a subtree contains exactly r points, which form a single group. Let $a_{s,j} \in \{1, \dots, K\}$ denote the cluster assignment of key k_j in subspace s , and let $\mathcal{C}_{s,i} = \{j : a_{s,j} = i\}$ be the index set of group i in subspace s .

Ball enclosing. Each group $\mathcal{C}_{s,i}$ is enclosed³ by a bounding ball $B(c_{s,i}, \rho_{s,i})$. The center is the mean of the group’s projected keys: $c_{s,i} = \frac{1}{|\mathcal{C}_{s,i}|} \sum_{j \in \mathcal{C}_{s,i}} \pi_s(k_j)$, and the radius is the maximum distance from the center to any group member:

$$\rho_{s,i} = \max_{j \in \mathcal{C}_{s,i}} \|\pi_s(k_j) - c_{s,i}\|_2. \quad (1)$$

For a halfspace query $H(q, \tau)$, the ball $B(c_{s,i}, \rho_{s,i})$ intersects $H(\pi_s(q), \tau_s)$ if and only if

$$\langle \pi_s(q), c_{s,i} \rangle + \rho_{s,i} \|\pi_s(q)\|_2 \geq \tau_s, \quad (2)$$

This calculation requires $O(d_s)$ operations per ball, regardless of the number of keys inside it. Groups whose balls do not satisfy (2) are pruned entirely with no false negatives (Challenge 1), since no point inside a non-intersecting ball can satisfy the threshold.

Index layout and memory. For each subspace s , the index stores: the K center vectors, the K radii, and the n assignment labels. The additional metadata is bounded by n/r descriptors per subspace — at most $1/r$ of the KV cache footprint (no more than 25% for $r = 4, S = 4$). Since the filter phase accesses only the $K = n/r$ centers, the full key array can be offloaded to slower memory tiers and fetched only for the surviving candidate set, reducing memory bandwidth pressure (see experiments in Section 5). The index construction has a time complexity of $O(S \cdot n \log(n/r))$.

The full procedures are provided in Algorithms 1 and 2 in Appendix I.

4.2 Query Processing

At each decoding step t , given a query $q_t \in \mathbb{R}^d$ and a threshold τ , the goal is to find $\mathcal{K}^* = \{k \in \mathcal{K} \mid \langle q_t, k \rangle \geq \tau\}$. LOUVER supports two query algorithms that differ in how the threshold is specified and how the index is traversed. Both share a common two-phase structure: a fast *index filter* that prunes

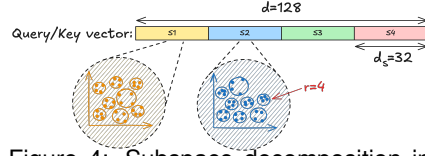


Figure 4: Subspace decomposition in LOUVER.

³See Definition 1 in Appendix G.

groups using cheap ball tests, followed by an *exact dot-product check* on the surviving candidates to enforce the threshold precisely. The threshold τ (or per-subspace thresholds $\tau_1, \dots, \tau_S$) can be provided by existing thresholding methods from the literature [31, 53, 49]. Our system also includes built-in threshold modules. We design and evaluate several thresholding strategies, which are discussed in detail in Appendix A.3.

Query 1: Full-subspace ball filter (per-subspace thresholds). This variant takes S per-subspace thresholds $\tau_1, \dots, \tau_S$ as input, where τ_s is a lower bound on $\langle \pi_s(q), \pi_s(k) \rangle$ for any relevant key k . Independently for each subspace s , a cluster i is *pruned* if its bounding ball does not intersect the halfspace $H(\pi_s(q), \tau_s)$:

$$\langle \pi_s(q), c_{s,i} \rangle + \rho_{s,i} \|\pi_s(q)\|_2 < \tau_s. \quad (3)$$

A key k is a candidate only if its cluster survives the ball test in *all* S subspaces. Candidates then undergo an exact dot-product check: only keys with $\langle q, k \rangle \geq \tau$ are included in the attention sum. Algorithm 3 (Appendix I) summarizes this procedure.

Query 2: TA filter (single global threshold).

This variant uses a single global threshold τ and adapts the Threshold Algorithm (TA) [15, 16] — originally a top- k retrieval algorithm — to our threshold-based setting.

The key observation is that the full dot product decomposes across subspaces: $\langle q, k \rangle = \sum_{s=1}^S \langle \pi_s(q), \pi_s(k) \rangle$. For any key k in cluster i of subspace s , the subspace contribution is bounded above by the ball: $\langle \pi_s(q), \pi_s(k) \rangle \leq \langle \pi_s(q), c_{s,i} \rangle + \rho_{s,i} \|\pi_s(q)\|_2 =: f_{s,i}$. Thus $f_{s,i}$ is an upper bound on how much any key in cluster i can contribute from subspace s .

The algorithm proceeds in rounds. *Initialization*: for each subspace s , compute $f_{s,i}$ for all K clusters and sort them in descending order; let $\sigma_s(d)$ denote the d -th cluster in this order. *Each round d* : visit the d -th cluster in every subspace simultaneously and add all their keys to a live set $\mathcal{L}$. After round d , the global upper bound on the full dot product of any key *not yet seen in any subspace* is $U(d) = \sum_{s=1}^S f_{s, \sigma_s(d)}$.

Stopping: scanning halts at the first depth d^* where $U(d^*) < \tau$. At this point, every key outside $\mathcal{L}$ belongs to a cluster ranked below d^* in every subspace, so its full dot product cannot exceed $U(d^*) < \tau$ and is safely pruned. Finally, keys in $\mathcal{L}$ undergo an exact dot-product check and those with $\langle q, k \rangle \geq \tau$ are included in the attention sum. Algorithm 4 (Appendix I) gives the full procedure.

Correctness. Both algorithms guarantee zero false negatives. For Query 1: if cluster i is pruned in any subspace s , then $\langle \pi_s(q), c_{s,i} \rangle + \rho_{s,i} \|\pi_s(q)\|_2 < \tau_s$, so no key inside the ball can achieve $\langle \pi_s(q), \pi_s(k) \rangle \geq \tau_s$, and hence cannot satisfy $\langle q, k \rangle \geq \tau$. For Query 2: at stopping depth d^* , any key $k \notin \mathcal{L}$ belongs to an unseen cluster in every subspace, so $\langle q, k \rangle \leq U(d^*) < \tau$. In both cases the final dot-product check is exact, so no false negatives are introduced. In our experiments, the index filter prunes up to 90% of keys before the exact check, meaning fewer than 10% of keys reach the dot-product computation (see Section 5).

4.3 Dynamic Updates

During decoding, a new key k_t is generated at every step. Rather than rebuilding the index from scratch, LOUVER maintains a fixed-size *buffer* $\mathcal{B}$ of recently generated keys not yet incorporated into the index. Keys in $\mathcal{B}$ are always attended densely, ensuring full recall regardless of buffer state.

Once $|\mathcal{B}|$ reaches B , an incremental update is triggered: the same index construction procedure is applied to the B buffered keys, producing B/r new groups whose centers, radii, and assignments are appended directly to the main index. The buffer is then reset. On GPU, this update runs concurrently with the attention computation, hiding its cost; on CPU it is amortized to $O(S \cdot d_s/r)$ per step — constant in n and negligible relative to dense attention. See Appendix J for implementation details.

Other System Details. Beyond the LOUVER index, our system (Figure 3) includes additional components: a *threshold selection* module that *automatically* identifies the relevance threshold (explained in Appendix A.3), and the *kernel-level implementations* optimized for GPU and CPU implementation in C++ with AVX/FP16 intrinsics (explained in Appendix K. As a result, as shown in Section 5, LOUVER *outperforms even highly optimized dense attention implementations such as FlashAttention* [12]. In Appendix H, we show that LOUVER addresses all design challenges introduced earlier.

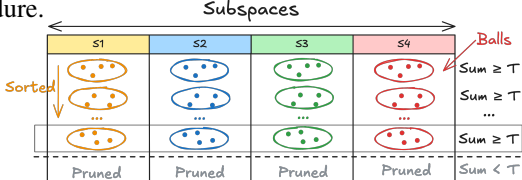


Figure 5: Visual illustration of the TA filtering procedure. Balls in each subspace are first sorted, and a cutoff is determined at the first level (row) where the aggregated score falls below the threshold.

Table 1: LongBench v1 at 10% KV retention, Llama-3.1-8B-Instruct. F1 (%).

Category	Method	NarrQA	Qasper	MFQ-EN	HotpotQA	2WikiMQA	MuSiQue	Avg
Dense	FlashAttention-2	16.4	46.4	55.0	54.5	47.1	30.8	41.7
Eviction	StreamingLLM	15.1	26.6	30.7	45.6	41.1	18.6	29.6
	H ₂ O	15.9	43.4	53.7	54.3	47.2	29.1	40.6
Retrieval	Quest	15.4	47.1	54.3	53.5	47.5	28.7	41.1
Adaptive	Twilight	15.7	46.7	55.0	54.5	48.2	29.7	41.6
Ours	LOUVER	16.2	47.1	55.2	54.5	47.1	30.9	41.8

Table 2: RULER at 32k tokens, 10% KV retention, Llama-3.1-8B-Instruct. Accuracy (%).

Category	Method	NIAH-S	NIAH-M	VT	Avg
Dense	FlashAttn-2	100.0	96.0	84.0	93.3
Eviction	StreamingLLM	0.0	9.3	0.0	3.1
	H ₂ O	100.0	96.0	52.0	82.7
Retrieval	Quest	100.0	96.0	60.0	85.3
Ours	LOUVER	100.0	98.0	74.0	90.7

Table 3: Reasoning benchmarks, DeepSeek-R1-Distill-Llama-8B. Accuracy (%).

Category	Method	AIME	MATH-500
Dense	FlashAttn-2	30.0	58.0
Eviction	—	—	—
	H ₂ O	—	55.0
Adaptive	Twilight ($p=0.85$)	25.0	44.0
Ours	LOUVER	30.0	62.0

5 Experiments

In this section, we show the performance of our proposed index, *Louver*, across multiple benchmarks and compare it under different scenarios and tasks. Additional details on experiments are provided in Appendix A. We evaluate on five open-weight instruction-tuned models (full details are provided in Appendix A.1): **Llama-3.1-8B-Instruct** [3], **DeepSeek-R1-Distill-Llama-8B** [19], **Llama-3.2-3B-Instruct** [3], **Qwen2.5-7B-Instruct** [47], and **Qwen2.5-14B-Instruct** [47]. Our evaluation spans both long-context understanding and reasoning-intensive tasks. For long-context evaluation, we use LongBench [5] and RULER [23]. For reasoning evaluation, we use AIME 2024 [36] and MATH-500 [37]. We compare *Louver* against three categories of baselines. *Fixed-budget* methods, including H₂O [52], StreamingLLM [45], and Quest [42]. *Adaptive-budget* methods, represented by Twilight [31]. *Retrieval-based offloading* methods that maintain a retrieval index in CPU memory and perform sparse attention on the GPU. This category includes RetrievalAttention [32], InfLLM [44], and MagicPIG [9]. As dense-attention references, we include Torch Eager/SDPA and FlashAttention-2 [12] on both GPU and CPU. The code is publicly available in <https://github.com/UIC-InDeXLab/Louver>.

Sparse Attention Accuracy Across Different Tasks. *Long-context benchmarks (fixed budget):* Tables 1 and 2 report results on LongBench and RULER using Llama-3.1-8B-Instruct under a fixed 10% KV budget. LOUVER estimates a retrieval threshold from a small reservoir sample to match the same retrieval budget as the baselines. LOUVER achieves the best overall performance, benefiting from its zero-false-negative guarantee: all keys with attention score above τ are retrieved, preventing important context tokens from being discarded. In contrast, fixed-fraction methods such as Twilight can still incur false negatives under the same budget constraint.

Long-output reasoning (adaptive budget): Table 3 reports results on AIME 2024 and MATH-500 using DeepSeek-R1-Distill-Llama-8B. Under continuously growing KV caches during chain-of-thought decoding, eviction and adaptive-budget baselines suffer from false negatives and accuracy degradation. In contrast, LOUVER-TA matches dense-attention accuracy on AIME 2024 and surpasses it on MATH-500, highlighting the importance of avoiding false negatives in long-output reasoning.

Long-output Decoding Latency. Figure 6 reports per-step attention latency during long reasoning decoding on AIME. Dense baselines and Twilight scale linearly with context length, since Twilight still computes full QK attention scores before pruning. In contrast, LOUVER prunes most keys before exact dot-product computation, resulting in sub-linear growth. LOUVER uses fused GPU and CPU kernels for efficient filtering and sparse attention execution. At 40k tokens, it achieves up to **15.3×** speedup over Torch Eager on GPU and **10.3×** over Torch SDPA on CPU, while maintaining dense accuracy on long-reasoning tasks.

Recall Guarantee: Zero False Negatives in Practice. We evaluate recall in two settings: ANN-based KV offloading methods and GPU-resident sparse attention methods. Recall is measured as the fraction of exact top- k attention keys recovered during decoding, with all methods compared

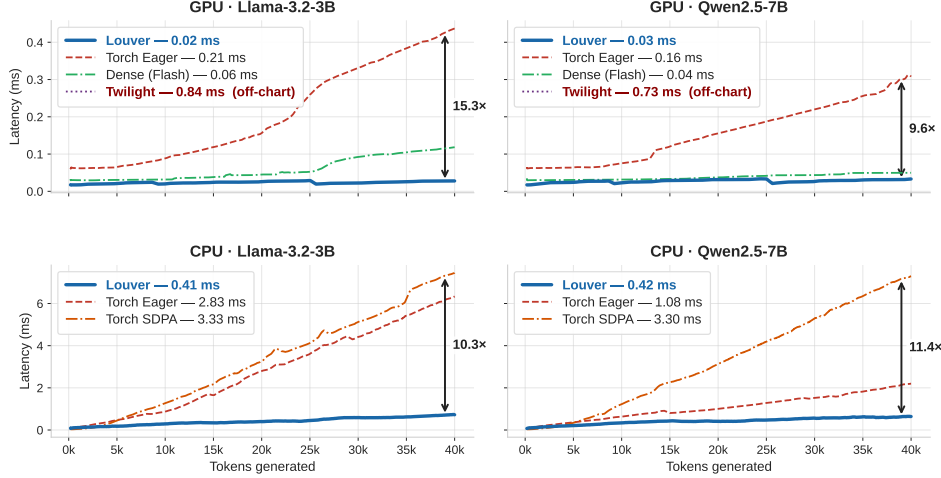


Figure 6: Per-step attention latency vs. context length. Top row: GPU; bottom row: CPU. Speedup annotations show LOUVER vs. the fastest dense baseline at 40k tokens.

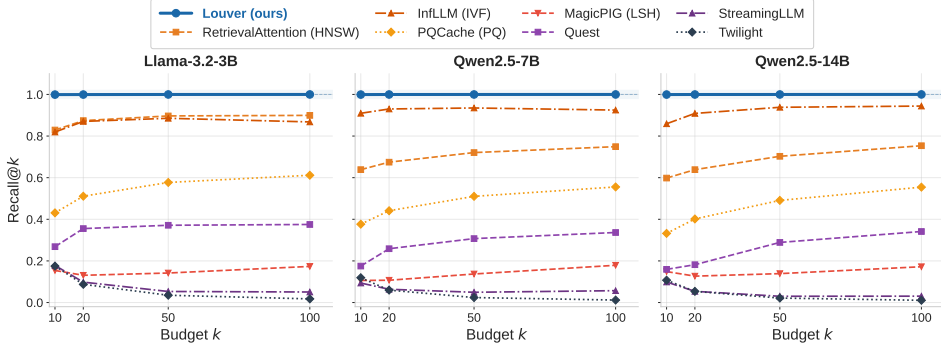


Figure 7: Recall@ k across three models. LOUVER achieves perfect recall ($\geq 99.9\%$) in all settings. Other methods fall substantially short.

under the same effective token budget. Figure 7 shows that LOUVER consistently achieves perfect recall ($\geq 99.9\%$) across all models and values of k , empirically validating its zero-false-negative guarantee. In contrast, ANN-based retrieval methods achieve only 60–93% recall, while sparse attention baselines perform substantially worse, often recovering below 40% of the true top- k keys.

KV Cache Offloading. In the CPU-offloading setting, all methods store the full KV cache in CPU memory and retrieve only a subset of tokens to GPU at each decoding step to reduce GPU memory usage. In this configuration, LOUVER keeps only parent cluster centers on GPU while storing all child entries (full KV pairs) on CPU, executing its range search filtering on GPU to filter balls. Table 4 shows that LOUVER achieves substantially higher accuracy (38.9% average F1) than all baselines ($\leq 26.2\%$). Its GPU-resident parent structure reduces retrieval latency to 0.07 ms per step while the more precise retrieval prunes over 75% of keys. Per-task results are provided in Appendix A.2.

Table 4: KV offloading on LongBench avg. F1 (%), Llama-3.1-8B-Instruct. Search time is index lookup per step; transfer time is CPU→GPU movement of retrieved KVs.

Method	Avg. F1 (%)	Search (ms)	Transfer (ms)	Extra GPU (MB, % of KV)
RetrievalAttention	25.1	53.95	7.05	0 (0%)
InfLLM	26.2	4.89	7.08	122 (12%)
MagicPIG	25.2	9.76	6.99	41 (4%)
LOUVER (OFFLOADED)	38.9	0.07	6.02	285 (28%)

Index design and threshold oracle. We ablate the index design and threshold-oracle variants, analyzing their impact on pruning efficiency, speedup, and accuracy–sparsity trade-offs. Increasing the number of subspaces improves pruning efficiency; with contiguous grouping and $S=16$, LOUVER scans only 16.3% of keys while achieving a 2.42 $\times$ speedup. Additional details are provided in Appendix G and Appendix A.3. Finally, a discussion on the limitations is provided in Appendix L.

6 Conclusion

We identify false negatives in sparse attention as a major source of accuracy degradation in long-context reasoning tasks. By reducing token selection to halfspace range searching, we develop LOUVER, an index with a provable zero-false-negative guarantee. LOUVER matches or exceeds dense-attention accuracy while achieving up to $15.3\times$ GPU and $10.3\times$ CPU speedups at 40k context length, outperforming existing KV offloading baselines. We hope this work highlights the importance of avoiding false negatives in sparse attention and provides a practical path toward efficient long-context inference.

References

- [1] Pankaj K Agarwal. Range searching. In *Handbook of discrete and computational geometry*, pages 1057–1092. Chapman and Hall/CRC, 2017.
- [2] Pankaj K Agarwal. Simplex range searching and its variants: A review. *A Journey Through Discrete Mathematics: A Tribute to Jiří Matoušek*, pages 1–30, 2017.
- [3] AI@Meta. The llama 3 herd of models, 2024. URL <https://arxiv.org/abs/2407.21783>.
- [4] Joshua Ainslie, James Lee-Thorp, Michiel De Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901, 2023.
- [5] Yushi Bai, Xin Lv, Jiajie Zhang, et al. Longbench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. URL <https://aclanthology.org/2024.acl-long.172/>.
- [6] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv.org*, 2020.
- [7] Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Baobao Chang, Junjie Hu, and Wen Xiao. Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069*, June 2024. doi: 10.48550/arXiv.2406.02069. URL <https://arxiv.org/abs/2406.02069>.
- [8] Renze Chen, Zhuofeng Wang, Beiquan Cao, Tong Wu, Size Zheng, Xiuhong Li, Xuechao Wei, Shengen Yan, Meng Li, and Yun Liang. Arkvale: Efficient generative llm inference with recallable key-value eviction. *Neural Information Processing Systems*, 37:113134–113155, 2024. doi: 10.52202/079017-3595.
- [9] Zhuoming Chen, Ranajoy Sadhukhan, Zihao Ye, Yang Zhou, Jianyu Zhang, Niklas Nolte, Yuandong Tian, Matthijs Douze, Léon Bottou, Zhihao Jia, et al. Magicpig: Lsh sampling for efficient llm generation. *International Conference on Learning Representations*, 2024. doi: 10.48550/arXiv.2410.16179.
- [10] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv.org*, 2019.
- [11] Douglas Comer. Ubiquitous b-tree. *ACM computing surveys (CSUR)*, 11(2):121–137, 1979.
- [12] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, July 2023. doi: 10.48550/arXiv.2307.08691. URL <https://arxiv.org/abs/2307.08691>.
- [13] Aditya Desai, Shuo Yang, Alejandro Cuadron, Ana Klimovic, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Hashattention: Semantic sparsity for faster inference. *arXiv.org*, 2024. doi: 10.48550/arXiv.2412.14468.
- [14] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library. 2024.

- [15] Ronald Fagin. Combining fuzzy information from multiple systems. In *Proceedings of the fifteenth ACM SIGACT-SIGMOD-SIGART symposium on Principles of database systems*, pages 216–226, 1996.
- [16] Ronald Fagin, Amnon Lotem, and Moni Naor. Optimal aggregation algorithms for middleware. In *Proceedings of the twentieth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*, pages 102–113, 2001.
- [17] Yuan Feng, Junlin Lv, Yukun Cao, Xike Xie, and S. Kevin Zhou. Ada-kv: Optimizing kv cache eviction by adaptive budget allocation for efficient llm inference. *arXiv preprint arXiv:2407.11550*, July 2024. doi: 10.48550/arXiv.2407.11550. URL <https://arxiv.org/abs/2407.11550>.
- [18] Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. Model tells you what to discard: Adaptive kv cache compression for llms. *International Conference on Learning Representations*, 2023. doi: 10.48550/arXiv.2310.01801.
- [19] Daya Guo, Dejian Yang, Haowei Zhang, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025. URL <https://arxiv.org/abs/2501.12948>.
- [20] Antonin Guttman. R-trees: A dynamic index structure for spatial searching. In *Proceedings of the 1984 ACM SIGMOD international conference on Management of data*, pages 47–57, 1984.
- [21] Sarel Har-Peled. *Geometric approximation algorithms*. Number 173. American Mathematical Soc., 2011.
- [22] Coleman Richard Charles Hooper, Sehoon Kim, Hiva Mohammadzadeh, Monishwaran Maheswaran, Sebastian Zhao, June Paik, Michael W Mahoney, Kurt Keutzer, and Amir Gholami. Squeezed attention: Accelerating long context length llm inference. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 32631–32652, 2025.
- [23] Cheng-Ping Hsieh et al. Ruler: What’s the real context size of your long-context language models?, 2024. URL <https://arxiv.org/abs/2404.06654>.
- [24] Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H. Abdi, Dongsheng Li, Chin-Yew Lin, et al. Minference 1.0: Accelerating pre-filling for long-context llms via dynamic sparse attention. *Neural Information Processing Systems*, 37:52481–52515, 2024. doi: 10.48550/arXiv.2407.02490.
- [25] Donghyeon Joo, Helya Hosseini, Ramyad Hadidi, and Bahar Asgari. Mustafar: Promoting unstructured sparsity for kv cache pruning in llm inference. *arXiv.org*, 2025. doi: 10.48550/arXiv.2505.22913.
- [26] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Chenyu Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*. ACM, 2023. doi: 10.1145/3600006.3613165. URL <https://arxiv.org/abs/2309.06180>.
- [27] Xunhao Lai, Jianqiao Lu, Yao Luo, Yiyuan Ma, and Xun Zhou. Flexprefill: A context-aware sparse attention mechanism for efficient long-sequence inference. *International Conference on Learning Representations*, 2025. doi: 10.48550/arXiv.2502.20766.
- [28] Haoyang Li, Yiming Li, Anxin Tian, Tianhao Tang, Zhanchao Xu, Xuejia Chen, Nicole Hu, Wei Dong, Qing Li, and Lei Chen. A survey on large language model acceleration based on kv cache management. *Trans. Mach. Learn. Res.*, 2024. doi: 10.48550/arXiv.2412.19442.
- [29] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. Snapkv: Llm knows what you are looking for before generation. *Neural Information Processing Systems*, 37:22947–22970, 2024. doi: 10.48550/arXiv.2404.14469.

- [30] Yuliang Li, Jianguo Wang, Benjamin Pullman, Nuno Bandeira, and Yannis Papakonstantinou. Index-based, high-dimensional, cosine threshold querying with optimality guarantees. *Theory of Computing Systems*, 65(1):42–83, 2021.
- [31] Chaofan Lin, Jiaming Tang, Shuo Yang, Hanshuo Wang, Tian Tang, Boyu Tian, Ion Stoica, Song Han, and Mingyu Gao. Twilight: Adaptive attention sparsity with hierarchical top- p pruning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- [32] Di Liu, Meng Chen, Baotong Lu, Huiqiang Jiang, Zhenhua Han, Qianxi Zhang, Qi Chen, Chengruidong Zhang, Bailu Ding, Kai Zhang, et al. Retrievalattention: Accelerating long-context llm inference via vector retrieval. *arXiv.org*, 2024. doi: 10.48550/arXiv.2409.10516.
- [33] Guangda Liu, Chengwei Li, Zhenyu Ning, Jing Lin, Yiwu Yao, Danning Ke, Minyi Guo, and Jieru Zhao. Freekv: Boosting kv cache retrieval for efficient llm inference. *arXiv.org*, 2025. doi: 10.48550/arXiv.2505.13109.
- [34] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhao Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for llm kv cache compression at test time. *Neural Information Processing Systems*, 36:52342–52364, 2023. doi: 10.48550/arXiv.2305.17118.
- [35] Yu A Malkov and Dmitry A Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2018.
- [36] Mathematical Association of America. American invitational mathematics examination (aime) 2024, 2024. URL <https://maa.org/>.
- [37] OpenAI. Math-500: A benchmark for measuring mathematical reasoning in language models. <https://github.com/openai/prm800k>, 2023.
- [38] Luka Ribar, Ivan Chelombiev, Luke Hudlass-Galley, Charlie Blake, Carlo Luschi, and Douglas Orr. Sparq attention: Bandwidth-efficient llm inference. *International Conference on Machine Learning*, 2023. doi: 10.48550/arXiv.2312.04985.
- [39] Prajwal Singhanian, Siddharth Singh, Shwai He, Soheil Feizi, and Abhinav Bhatele. Loki: Low-rank keys for efficient sparse attention. *Neural Information Processing Systems*, 37: 16692–16723, 2024. doi: 10.48550/arXiv.2406.02542.
- [40] Hanshi Sun, Li-Wen Chang, Wenlei Bao, Size Zheng, Ningxin Zheng, Xin Liu, Harry Dong, Yuejie Chi, and Beidi Chen. Shadowkv: Kv cache in shadows for high-throughput long-context llm inference. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 57355–57373. PMLR, 2025. URL <https://proceedings.mlr.press/v267/sun25b.html>.
- [41] Yutao Sun, Zhenyu Li, Yike Zhang, Tengyu Pan, Bowen Dong, Yuyi Guo, and Jianyong Wang. Efficient attention mechanisms for large language models: A survey. *arXiv.org*, 2025. doi: 10.48550/arXiv.2507.19595.
- [42] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context llm inference. *International Conference on Machine Learning*, 2024. doi: 10.48550/arXiv.2406.10774.
- [43] Emo Welzl. Partition trees for triangle counting and other range searching problems. In *Proceedings of the fourth annual symposium on Computational geometry*, pages 23–33, 1988.
- [44] Chaojun Xiao, Pengle Zhang, Xu Han, Guangxuan Xiao, Yankai Lin, Zhengyan Zhang, Zhiyuan Liu, and Maosong Sun. Inllm: Training-free long-context extrapolation for llms with an efficient context memory. *Neural Information Processing Systems*, 37:119638–119661, 2024. doi: 10.52202/079017-3801.

- [45] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *International Conference on Learning Representations*, 2023. doi: 10.48550/arXiv.2309.17453.
- [46] Ruyi Xu, Guangxuan Xiao, Haofeng Huang, Junxian Guo, and Song Han. Xattention: Block sparse attention with antidiagonal scoring. *International Conference on Machine Learning*, 2025. doi: 10.48550/arXiv.2503.16428.
- [47] An Yang, Baosong Yang, Beichen Zhang, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. URL <https://arxiv.org/abs/2412.15115>.
- [48] Dongjie Yang, Xiaodong Han, Yan Gao, Yao Hu, Shilin Zhang, and Hai Zhao. Pyramidinfer: Pyramid kv cache compression for high-throughput llm inference. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, 2024. URL <https://aclanthology.org/2024.findings-acl.195/>.
- [49] Jiayi Yuan, Cameron Shinn, Kai Xu, Jingze Cui, George Klimiashvili, Guangxuan Xiao, Perkz Zheng, Bo Li, Yuxin Zhou, Zhouhai Ye, Weijie You, Tian Zheng, Dominic Brown, Pengbo Wang, Richard Cai, Julien Demouth, John D. Owens, Xia Hu, Song Han, Timmy Liu, and Huizi Mao. Blasst: Dynamic blocked attention sparsity via softmax thresholding. *arXiv preprint arXiv:2512.12087*, December 2025. doi: 10.48550/arXiv.2512.12087. URL <https://arxiv.org/abs/2512.12087>.
- [50] Hailin Zhang, Xiaodong Ji, Yilin Chen, Fangcheng Fu, Xupeng Miao, Xiaonan Nie, Weipeng Chen, and Bin Cui. Pqcache: Product quantization-based kvcache for long context llm inference. *Proc. ACM Manag. Data*, 3(3):1–30, 2024. doi: 10.1145/3725338.
- [51] Jintao Zhang, Chendong Xiang, Haofeng Huang, Jia Wei, Haocheng Xi, Jun Zhu, and Jianfei Chen. Spargeattention: Accurate and training-free sparse attention accelerating any model inference. *International Conference on Machine Learning*, 2025.
- [52] Zhenyu (Allen) Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark W. Barrett, et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Neural Information Processing Systems*, 36:34661–34710, 2023. doi: 10.48550/arXiv.2306.14048.
- [53] Kan Zhu, Tian Tang, Qinyu Xu, Yile Gu, Zhichen Zeng, Rohan Kadekodi, Liangyu Zhao, Ang Li, Arvind Krishnamurthy, and Baris Kasikci. Tactic: Adaptive sparse attention with clustering and distribution fitting for long-context llms. *arXiv preprint arXiv:2502.12216*, 2025.
- [54] Qianchao Zhu, Jiangfei Duan, Chang Chen, Siran Liu, Xiuhong Li, Guanyu Feng, Xin Lv, Huanqi Cao, Chuanfu Xiao, Xingcheng Zhang, Dahua Lin, and Chao Yang. Sampleattention: Near-lossless acceleration of long context llm inference with adaptive structured sparse attention. *arXiv preprint arXiv:2406.15486*, June 2024. doi: 10.48550/arXiv.2406.15486. URL <https://arxiv.org/abs/2406.15486>.

Appendix

Table of Content

1. Additional Experiments	A
2. Related Work	B
3. Experimental Setup: False-Negativity Sensitivity	C
4. Experimental Setup: Per-Step Score Distribution	D
5. Key Norms Are Not Fixed	E
6. Index Design Challenges	F
7. Ablation on Index Design	G
8. How Louver Addresses the Design Challenges	H
9. Pseudo-codes	I
10. Update Process	J
11. Kernel Implementation Details	K
12. Discussion	L

A Additional Experiments

A.1 Experimental Setup

Models. We use five open-weight models, all loaded in BF16 on a single GPU. **Llama-3.1-8B-Instruct** [3] (128k context window) is used for accuracy on long-input benchmarks (LongBench, RULER) and offloading experiments. **DeepSeek-R1-Distill-Llama-8B** [19], distilled from DeepSeek-R1 to produce extended chain-of-thought reasoning traces, is used for long-output benchmarks (AIME 2024, MATH-500). **Llama-3.2-3B-Instruct** [3] and **Qwen2.5-7B-Instruct** [47] are used for latency, recall, and index ablation experiments. **Qwen2.5-14B-Instruct** [47] is additionally included in the recall experiment.

Benchmarks. *Long-input benchmarks* test retrieval of relevant information from a long prefill context. **LongBench v1** [5] is a multi-task question-answering benchmark covering NarrativeQA, Qasper, MultiFieldQA-EN, HotpotQA, 2WikiMultiHopQA, and MuSiQue; we report average F1. **RULER** [23] is a synthetic benchmark with controlled sequence lengths that isolates retrieval difficulty; we evaluate on needle-in-a-haystack single (NIAH-S), multi-needle (NIAH-M), and variable tracking (VT) at 32k tokens. *Long-output benchmarks* test accuracy under extended chain-of-thought generation, where the KV cache grows throughout decoding. **AIME 2024** [36] consists of 30 competition mathematics problems; we report pass@1 accuracy. **MATH-500** [37] is a curated set of 500 problems from the MATH benchmark; we report accuracy.

Baselines. *Fixed-budget.* **H₂O** [52] is eviction-based: it retains heavy-hitter tokens by cumulative attention score plus a recency window. **StreamingLLM** [45] is eviction-based: it keeps a fixed set of attention sink tokens plus a sliding recency window. **Quest** [42] is retrieval-based: it scores page-level KV chunks via a $\text{sign}(q) \cdot \max(k)$ proxy and retrieves the top- k chunks. All three operate at a 10% KV budget (fraction of the full context length).

Adaptive-budget. **Twilight** [31] is retrieval-based: it computes full QK attention scores at each step, softmax-normalizes them, and retains the smallest set of tokens whose cumulative probability mass reaches $p = 0.85$. We use the same threshold $p = 0.85$ for LOUVER-TA, making the budget directly comparable.

Retrieval-based offloading. All three methods offload the full KV cache to CPU and retrieve a budget of tokens per step via an ANN index. **RetrievalAttention** [32] builds an HNSW index on CPU and fetches approximate nearest-neighbor KV pairs on demand. **InfLLM** [44] uses IVF clustering on CPU and retrieves the top-scoring clusters per step. **MagicPIG** [9] uses LSH on CPU to identify relevant KV pairs via hash bucket lookup. All three retrieve 15% of stored tokens per step.

Dense attention. **Torch Eager** uses the standard unfused $O(N)$ implementation. **FlashAttention-2** [12] is the IO-aware fused kernel and our primary dense accuracy upper bound. **Torch SDPA** dispatches to the most efficient available backend (FlashAttention-2 or memory-efficient attention) and is used as the CPU dense reference in latency experiments.

System configuration. *Accuracy experiments* (LongBench, RULER, AIME 2024, MATH-500, of-flooding) are run on the Delta cluster (NCSA) on a single-GPU node equipped with one **NVIDIA A100 40 GB SXM4** (Ampere, 40 GB HBM2e, 1.6 TB/s memory bandwidth), 64 CPU cores (AMD EPYC 7763), and 256 GB of RAM.

Latency and recall experiments are run on a local workstation with an **NVIDIA RTX 5090** (32 GB), an **AMD Ryzen Threadripper 7970X** (64 threads), and 256 GB DDR5. CPU latency experiments use the Threadripper with the GPU idle. All models are loaded in BF16; CUDA 12.4, PyTorch 2.5, and Triton 3.1 are used throughout.

A.2 KV Cache Offloading: Per-Task Breakdown

Table 5 reports per-task LongBench F1 scores for all offloading methods at a 15% token budget on Llama-3.1-8B-Instruct. LOUVER leads on every task, with the largest gains on multi-hop reasoning tasks (MuSiQue, Qasper) that require precise retrieval of specific evidence tokens — exactly the setting where its zero-false-negative guarantee matters most.

Table 5: Per-task LongBench F1 (%) for KV offloading methods at 15% budget, Llama-3.1-8B-Instruct.

Category	Method	HotpotQA	2WikiMQA	MuSiQue	Qasper	NarrativeQA	Avg
Offloading	RetrievalAttention (HNSW)	41.0	27.4	21.3	17.9	18.0	25.1
	InfLLM (IVF)	51.0	19.4	20.0	19.9	20.5	26.2
	MagicPIG (LSH)	42.0	24.2	20.0	17.5	22.3	25.2
Ours	LOUVER	59.0	32.5	35.0	36.3	31.9	38.9

A.3 Threshold Oracle Comparison

LOUVER is agnostic to how the threshold τ is obtained and accepts any oracle as input, including methods from prior work [31, 53, 49]. We additionally provide three built-in sampling-based oracles that maintain a reservoir of past keys during decoding and estimate τ from dot products against the current query; the reservoir is updated in $O(1)$ per step via reservoir sampling.

- **Sample-Max:** τ = maximum dot product of the query against the reservoir. Conservative; tends to over-include.
- **Sample-MeanMax:** τ = average of sample-max and sample-mean scores. Less conservative, tighter candidate sets.
- **Sample-Gap:** cut at the largest gap between consecutive sorted reservoir scores. Adapts to natural score distribution discontinuities.

For the ablation below we additionally evaluate **sample-topk**(m) (τ = m -th largest reservoir score, $m \in \{2, 5, 10\}$) and **budget**(α) (τ = $(1-\alpha)$ -quantile of reservoir, targeting retrieval fraction α), giving nine variants in total. All are estimated from a reservoir of size 256.

Attention concentration. Attention distributions during long-output decoding are highly concentrated: on average, only **3.9%** of keys are needed to cover 50% of the total softmax mass, with low variance (std 0.87%) across 2000 decode steps on DeepSeek-R1-Distill-Qwen-14B (see also Figure 1c in the main paper). This concentration justifies threshold-based retrieval: the important tokens form a stable, small set that a reservoir-estimated τ can reliably identify.

Oracle stability. Table 6 reports the mean and standard deviation of τ across 2000 decode steps (context length 772–2770 tokens, top-25% layers, DeepSeek-R1-Distill-Qwen-14B). All oracles produce stable thresholds (CoV $\leq 25\%$); budget-based oracles have higher variance because they track the quantile of a shifting distribution, while sample-gap and sample-max are anchored to the reservoir’s extremes and are more stable. Figure 8 shows τ trajectories over decode steps for each oracle.

B Related Work

Inference-time Sparse Attention and KV Cache Management. Sparse attention for long-context inference can be broadly divided into *eviction-based* and *retrieval-based* approaches. Eviction-based methods permanently remove tokens from the KV cache to reduce memory or bandwidth cost. Earlier

Table 6: Threshold oracle comparison: mean and std of τ over 2000 decode steps. DeepSeek-R1-Distill-Qwen-14B, reservoir size 256, top-25% layers averaged.

Oracle	τ mean	τ std	CoV	Description
sample-max	62.0	5.3	8.5%	max of reservoir
sample-topk(2)	52.8	5.6	10.6%	2nd largest in reservoir
sample-gap	51.1	5.5	10.8%	largest gap in reservoir
sample-topk(5)	44.8	5.7	12.7%	5th largest in reservoir
sample-topk(10)	38.6	5.7	14.8%	10th largest in reservoir
sample-mean-max	31.3	4.6	14.6%	mean of per-head maxima
budget(5%)	36.8	5.7	15.6%	95th-pct of reservoir
budget(10%)	29.2	5.8	19.8%	90th-pct of reservoir
budget(15%)	24.2	5.9	24.4%	85th-pct of reservoir

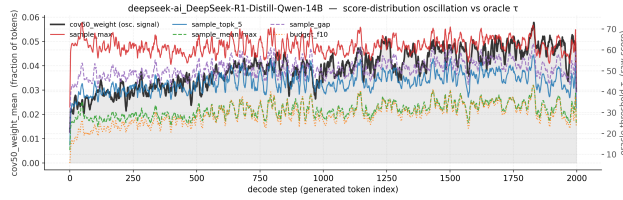


Figure 8: Threshold τ trajectories for each oracle variant over 2000 decode steps (DeepSeek-R1-Distill-Qwen-14B). Sample-based oracles (sample-max, sample-gap) are most stable; budget oracles show higher variance as the score distribution shifts.

approaches rely on fixed sparsity patterns such as sliding windows or attention sinks [10, 6, 45], while more recent work dynamically selects tokens to discard using runtime statistics or importance estimates, including SnapKV [29], H₂O [52], FastGen [18], Scissorhands [34], and Mustafar [25]. Some methods instead focus on reducing attention cost during prefilling over long inputs, including MInference [24], FlexPrefill [27], and XAttention [46]. A common limitation of eviction-based approaches is irreversibility: once tokens are removed or compressed, they cannot be recovered later if they become relevant.

Retrieval-based methods preserve the full KV cache logically and select relevant entries dynamically at inference time. Some systems offload the KV cache to lower tiers of the memory hierarchy and retrieve entries on demand, including RetrievalAttention [32], MagicPIG [9], PQCache [50], ArkVale [8], InfLLM [44], FreeKV [33], and ShadowKV [40]. Others keep all entries on-device but reduce memory bandwidth by selecting only a subset of important keys during attention computation, including SparQ [38], Quest [42], HashAttention [13], Loki [39], SparseAttention [51], and Squeezed Attention [22]. Our work falls in this category: we formulate sparse attention as a range-search problem over key vectors and introduce Louver, a dynamically updated index supporting exact thresholded retrieval with *zero false negatives* during decoding. It also generalizes to both offloading and non-offloading settings.

KV Cache Indexing. Retrieval-based sparse attention methods differ primarily in how they index the KV cache. Several cast the selection problem as approximate nearest neighbor search (ANN) or maximum inner product search (MIPS). MagicPIG [9] uses locality-sensitive hashing (LSH), PQCache [50] uses product quantization (PQ), and RetrievalAttention [32] constructs an attention-aware vector retrieval index tailored to the distribution mismatch between queries and cached keys. HashAttention [13] instead uses learned binary signatures in Hamming space for efficient token selection. Squeezed Attention [22] builds a hierarchical centroid-based index for fixed prompt contexts constructed during prefilling. In contrast, our approach performs *range searching* over key vectors without requiring normalization or dealing with query-key distributional mismatch [32], and supports incremental updates during decoding.

Adaptive Sparsity Budgets. Most sparse attention methods select a fixed number of tokens per query. Recent work explores adaptive sparsity budgets based on attention mass or coverage criteria. Twilight [31] selects tokens until cumulative attention mass exceeds a threshold, Tactic [53] uses coverage-based stopping criteria, and SampleAttention [54] adaptively identifies a minimal structured subset of key-value pairs. Per-layer and per-head budget allocation are explored by PyramidInfer [48],

PyramidKV [7], and Ada-KV [17]. BLASST [49] instead prunes low-importance attention blocks inside tiled hardware-optimized attention kernels using calibrated thresholds. While these works address the same challenge for a fixed-size formulation, they don’t guarantee the *zero false negativity*. Our approach differs in that it naturally supports dynamic, per-query thresholds that vary across heads, layers, and decoding steps, while retrieving *all keys* satisfying the threshold. This is a result of designing an index that handles *range searching* queries.

Complementary Attention Acceleration Techniques. Hardware-aware exact-attention kernels such as FlashAttention [12] and serving systems such as vLLM [26] improve dense attention efficiency and are complementary to sparse retrieval methods. Similarly, linear-complexity attention architectures and KV cache quantization techniques [41, 28] address different aspects of long-context inference and can be combined with our approach.

C Experimental Setup: False-Negative Sensitivity (Observation 1)

Data and prompt. We construct a synthetic summation task that gives us full control over which context tokens are *relevant* (i.e., must be attended to in order to produce the correct answer). Each instance is a list of N integers drawn uniformly from $\{1, 2, 3\}$, formatted as the prompt “*Consider the list of numbers: $x_1, x_2, \dots, x_N$. What is the sum of the numbers? Answer with just one integer and nothing else. The answer is*”. Greedy decoding produces the answer token, which is compared against $\sum_i x_i$. We tokenize the prompt with the model’s native tokenizer and record, for each number x_i , the set of token positions that span its surface form; these positions form the *relevant* set R . Filler tokens matching the closed-class set $\{\text{“the”, “of”, “a”, “an”, “and”, “with”, “to”, “in”, “on”, “just”, “one”, “nothing”, “else”, “this”, “;”, “:”, “.”}\}$ form the *irrelevant* set $\bar{R}$.

Models and sparse-attention simulation. We evaluate Llama-3.2-1B/3B-Instruct, Llama-3.1-8B-Instruct, and Qwen2.5-3B/7B-Instruct in fp16 with eager attention. Sparse attention is simulated by masking selected *key* positions: we run a single forward pass with a 4-D additive attention mask of shape $(1, 1, S, S)$ that combines the standard causal mask with $-\infty$ entries on the columns of dropped positions, so that no query attends to those keys. This is mathematically equivalent to evicting those KV-cache entries while keeping all model weights and remaining keys/values intact. For Figure 1a (Experiment 1) we drop $M=1$ token at a time, comparing one randomly chosen token from R with one randomly chosen token from $\bar{R}$, with $N=8$ and 30 random lists per model. For Figure 1b (Experiment 3) we sweep $N \in \{2, 3, 4, 5, 6, 8, 10, 12, 16\}$ and a fixed- K budget $K \in \{2, 3, 4, 6\}$ that retains exactly K of the N number tokens (chosen uniformly at random) plus structural context (BOS, the question span, and all non-list tokens), and compare against the dense baseline and an oracle “keep-all-relevant” policy that retains every token in R . Each configuration is averaged over 20 random lists with seed 2.

Metrics. We use two complementary metrics that together capture both decoding behaviour and full distributional shift. (i) $P(\text{answer changed vs. dense})$ is the fraction of trials in which greedy decoding under the dropped mask produces a different output string than greedy decoding under dense attention, when both are run for 4 tokens; this is the metric reported in Figure 1a. (ii) $\text{KL}(p_{\text{drop}} \parallel p_{\text{dense}})$ is the Kullback–Leibler divergence between the next-token distributions obtained from the dense and dropped forward passes after softmax over the full vocabulary; this is the metric reported in Figure 1b. Both are averaged across trials per (N, method) cell. Standard errors on the bar plot use the binomial estimator $\sqrt{p(1-p)/n}$.

D Experimental Setup: Per-Step Score Distribution (Observation 2)

Model, prompt, and generation. We use DeepSeek-R1-Distill-Qwen-14B loaded in fp16 with eager attention, decoded greedily for 2,000 new tokens on a single multi-part reasoning prompt. The prompt asks the model to solve a six-part combinatorial scheduling problem (assigning 7 cities to 5 days under seven explicit constraints, costing the resulting schedules, deriving a counting recurrence, identifying equivalence classes, writing pseudocode for a backtracking solver, and recomputing counts under perturbed constraints). Two stylistic instructions are pinned in the system prompt to elicit representative reasoning behaviour: (A) the model must produce an explicit numbered plan and at least four “pivot” blocks beginning with “Wait,” / “Actually,” / “Hmm, let me reconsider –”, and (B) every numeric quantity must be re-derived inline as LHS = a OP b = result. These instructions induce both wide-tail synthesis steps (pivots, plan back-references, recaps) and narrow-tail local steps

(intermediate arithmetic, template scaffolding) within the same trace, which is what the cov50 metric is designed to discriminate.

Q/K capture and metric. During decoding we hook every attention layer and persist, for each generated position t , the post-RoPE query vector $q_{t,h} \in \mathbb{R}^d$ for every query head h , alongside the full prefix of post-RoPE keys $k_{1:T_t,h'}$ for every key/value head h' , where T_t is the prompt length plus t . The snapshot is replayed offline. For each layer we expand the GQA mapping $h \mapsto h' = h \bmod (\text{num_kv_heads})$ and compute attention scores in fp32 as $s_{t,h,j} = q_{t,h}^\top k_{j,h'}/\sqrt{d}$ for $j \leq T_t$, then weights $w_{t,h,j} = \text{softmax}_j(s_{t,h,j})$. The per-head, per-step cov50 statistic is the smallest k such that the top- k cumulative mass of $(w_{t,h,j})_j$ reaches 0.5, normalised by T_t :

$$\text{cov50}_{\text{weight}}(t, h) = \frac{1}{T_t} \min \left\{ k : \sum_{j \in \text{top}_k(w_{t,h,\cdot})} w_{t,h,j} \geq 0.5 \right\}.$$

A score-space variant $\text{cov50}_{\text{score}}$ is computed identically after shifting the raw scores by their per-step minimum and renormalising to a distribution; we report the weight-space variant in Figure 1c. Cov50 is bounded in $(0, 1]$ and is comparable across decoding positions of different lengths T_t : small values indicate that attention concentrates on a tiny fraction of the prefix, while large values indicate diffuse attention.

Aggregation and figure. Computation runs on the GPU, layer by layer, with chunking over the query dimension to bound memory. For each step t we aggregate $\text{cov50}_{\text{weight}}(t, h)$ across all heads of all layers by taking the mean, yielding the time-series plotted in Figure 1c. The prefill region ($t < \text{prompt length}$) is shown faded for reference; only the decode region is annotated. To ground the visual signal, we automatically extract decode-step windows corresponding to the four highest and four lowest cov50 values (separated by at least 120 tokens to avoid clustering) and decode the ± 20 -token neighbourhood of each peak.

Decoded windows. We list the eight extracted windows below; the bracketed token `[[...]]` marks the target step t , and the value next to each label is the mean cov50 at that step.

Wide-tail (transition / synthesis) windows:

- w1 ($t=1365$, cov50=0.058). “that would make Mon have two cities: G and A. Is that allowed? Constraint (f) `[[says]]` at most 2 per day, so yes. Wait, but in this subcase, G is”
- w2 ($t=1460$, cov50=0.060). “can’t be on Tue because that would make three cities on Tue (C, F, B). `[[Wait]]`, no, each day can have up to 2 cities. So if B is on Tue,”
- w3 ($t=1648$, cov50=0.054). “we have 7 cities: A, B, C, D, E, F, G. `[[All]]` are assigned except Fri. Wait, no, Fri is empty. So we need to assign the remaining”
- w4 ($t=1767$, cov50=0.056). “because we have to assign all cities, but Fri is empty. So this is invalid. Wait, `[[no]]`, Fri can have zero cities, but the problem says each city has exactly one delivery, so all”

Narrow-tail (local / template) windows:

- N1 ($t=1050$, cov50=0.019). “Fri is available. So, four possibilities for G and F. Let me consider each subcase `[[.]]` Subcase 1a: G on Mon, F on Tue”
- N2 ($t=1265$, cov50=0.029). “A can’t be on Thu. So, possible A assignments: Mon, Tue, Wed. Let `[[’s]]` try A on Mon. Then B can be on Tue, Wed, Thu.”
- N3 ($t=1557$, cov50=0.022). “has D and B. That’s allowed. So, two possibilities for B. Let’s explore both `[[.]]` Subsubcase 1a1: A on Mon, B on Wed”
- N4 ($t=1822$, cov50=0.029). “cities except Fri, which is empty. That’s a problem. So this subsubcase is invalid `[[.]]` Wait, no, let me recount: Mon: G, A (2 cities), Tue: C”

Discussion. The two clusters expose qualitatively distinct attention regimes within the *same* decoding trace. Wide-tail steps consistently fire at points where the model must integrate information from far-apart spans of the prefix: checking a freshly-emitted partial schedule against constraint (f) declared hundreds of tokens earlier (w1), pivoting away from a wrong intermediate conclusion by re-reading the per-day capacity rule (w2), recapping the seven-city universe before reasoning about the remaining slots (w3), or revisiting whether “Fri empty” is actually permitted (w4). All four contain the model’s characteristic “Wait, no...” pivot phrase and force attention to spread over many earlier tokens to reconcile the local state with global constraints. Narrow-tail steps, by contrast, sit at template boundaries where the next token is essentially determined by the most recent few words: the period that closes a clause and opens a new **Subcase** block (N1, N3), the contraction “Let’s” that introduces a fresh local trial (N2), or the period that closes “... is invalid” before backtracking (N4). Cov50 values differ by roughly a factor of three between the two regimes (~ 0.02 vs. ~ 0.06), and the alternation between them happens on a scale of tens to a few hundred decode steps. Any sparse-attention policy with a fixed token budget either over-provisions on narrow steps (wasting compute) or under-provisions on wide steps (incurring false negatives on the exact tokens the model is trying to reconcile), which is the empirical basis for the dynamic, recall-oriented formulation argued for in the main text.

E Key Norms Are Not Fixed

Setup. We run Qwen2.5-7B-Instruct (fp16, eager attention) on a short technical prompt and decode 128 greedy tokens, hooking every attention layer to record the post-RoPE key vector $k_{\ell,h,t}$ for each layer ℓ , KV-head h , and token position t . We then compute the L2 norm $\|k_{\ell,h,t}\|_2$ and study how it varies across the three axes. Aggregated over all $(L, H, T) = (28, 4, 193)$ entries the norm spans $[0.37, 923]$ with mean 40.0 and standard deviation 100.1; the max / min ratio is $\approx 2.5 \times 10^3$, i.e. keys are far from unit-normalised. Marginalising one axis at a time, the per-layer mean still varies by $17\times$, the per-head mean by $2.5\times$, and the per-position mean by $1.4\times$, so the variation is not noise but structured along each axis. Figure 9 shows the within-layer distribution and makes the depth dependence explicit; Figure 10 shows the mean and $\pm 1\sigma$ envelope at each position, revealing systematic spikes at the BOS / system-header tokens and around the prompt $\rightarrow$ decode boundary.

Implication for sparse attention. Attention scores $s_{t,j} = q_t^\top k_j / \sqrt{d}$ scale linearly with $\|k_j\|$, so the norm is a real component of relevance: a key with large $\|k\|$ contributes more mass under any query whose direction is not orthogonal to it. Index structures that normalise keys away (cosine similarity, MIPS reductions to angular search) therefore discard a signal the model itself produced; in contrast, half-space range searching on the raw key vectors preserves the magnitude and treats the score threshold T in the units the attention layer uses. This is one of the reasons we model sparse attention as range searching rather than as nearest-neighbour or cosine retrieval.

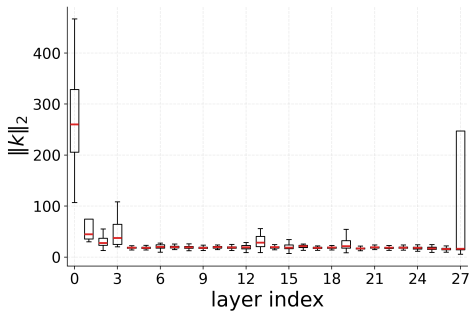


Figure 9: Per-layer distribution of key-vector L2 norms across all KV-heads and token positions in a 128-token Qwen2.5-7B trace. The within-layer spread is wide and the median grows substantially with depth, so a single global norm assumption is not justified.

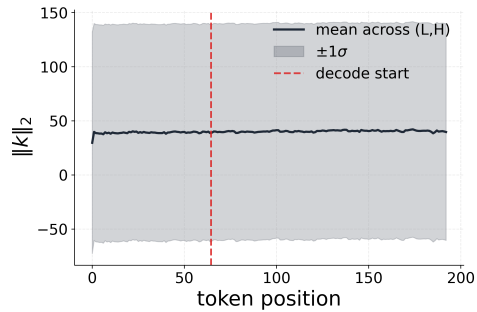


Figure 10: Mean key-norm at each token position (averaged across all $L \times H$ heads) with a $\pm 1\sigma$ envelope. The dashed line marks the prompt $\rightarrow$ decode boundary. Norms peak at the first few positions (BOS / system header) and exhibit position-dependent drift rather than a constant value.

F Index Design Challenges

Following the reduction of Problem 1, our goal is to design an index over the KV cache that supports efficient halfspace range searching queries. Classical data structures for range searching include geometric indexes such as KD-trees, range trees, partition trees [43], and R-trees [20], which have been extensively studied in the literature (see the survey [1]). In contrast, production database systems primarily rely on B-tree variants, in particular B+-trees [11], which are widely used for range queries over ordered data. However, these approaches are not well-suited to the LLM inference setting. KV cache indexing must operate under strict latency constraints during decoding, avoid introducing significant overhead, and be compatible with GPU-oriented execution. Moreover, the index must support dynamic updates, as new keys are generated and inserted at each decoding step.

We outline the key challenges of KV cache index design below. Then, in the next section, we present our index, LOUVER, which addresses these challenges.

Challenge 1 (High Pruning Power with Full Recall). The primary goal of an index for halfspace range searching is to achieve sublinear query time by avoiding a full scan over all keys. A brute-force approach evaluates all $t - 1$ keys in the KV cache and returns those satisfying the threshold τ . To improve upon this, the index must be able to efficiently **prune** large subsets of keys using cheap computations. At the same time, the index must guarantee full recall, i.e., zero false negatives. This requirement is critical in the LLM setting as discussed before. Therefore, the main challenge is to prune a large number of irrelevant keys while still retrieving all relevant ones. In addition, the pruning strategy should work well for the actual distribution of key and query vectors in transformer attention, so the index must be designed specifically for LLM inference.

Challenge 2 (Dynamic and Query-Adaptive). The index must be dynamic. During decoding, new keys are generated at every step, and in long-output tasks such as reasoning, most keys are produced on-the-fly during autoregressive generation rather than during prefilling. Therefore, the index must support efficient updates, allowing new keys to be inserted continuously without degrading performance. In addition, the index must be query-adaptive. As observed earlier in Figure 1c, the number of relevant keys varies across decoding steps and depends on the query. The range searching formulation naturally supports this by allowing arbitrary thresholds at query time.

Challenge 3 (Efficiency and Integration). The final challenge is efficiency. The index should not introduce additional overhead during inference, especially in the decoding phase, which is already the main bottleneck. It should also avoid increasing the memory footprint of the KV cache, as memory usage is another critical constraint. In addition, the index should make effective use of the underlying hardware, including both CPU and GPU (CUDA) environments, and help reduce memory bandwidth during attention computation—one of the main goals of sparse attention. It should also achieve speedups over highly optimized dense attention implementations such as FlashAttention [12]. Finally, the index should be easy to integrate into existing LLM inference systems, requiring minimal changes and low implementation overhead.

In Appendix G, we present a general class of index structures with the potential for achieving strong pruning while maintaining full recall (addressing Challenge 1) and evaluate them through ablation studies on real LLM key and query distributions. The resulting insights guide the design of our final index, LOUVER, presented in the next section.

G Ablation on Index Design

G.1 High Pruning Power with Full Recall

Achieving high pruning power while preserving zero false negatives requires the index to prune entire groups of keys cheaply without inspecting every key individually. Inspired by classical geometric range-searching indexes [21, 1], we define a general family of indexes parameterized by two choices: an *enclosing geometry* and a *grouping strategy*.

Enclosing. The key idea is to *enclose* a subset of keys within a convex geometric shape E (e.g., a bounding ball, an axis-aligned bounding box (AABB), or an ellipsoid). At query time, given a halfspace $H(q, \tau) = \{k \in \mathbb{R}^d : \langle q, k \rangle \geq \tau\}$, we test whether E intersects H . If $E \cap H = \emptyset$, then no key inside E can satisfy the threshold, and the entire group is pruned with zero false negatives.

Definition 1 (Enclosing). An enclosing family $\mathcal{E}$ is a collection of convex sets in $\mathbb{R}^d$ such that for any point set $P \subset \mathbb{R}^d$ and any $E \in \mathcal{E}$ with $P \subseteq E$:

$$E \cap H = \emptyset \implies P \cap H = \emptyset.$$

This guarantee follows directly from convexity: if the enclosing shape does not intersect the halfspace, no point it contains can either. The key practical requirement is that the intersection test $E \cap H \stackrel{?}{=} \emptyset$ be *cheaper* than evaluating all $|P|$ dot products individually. For a bounding ball $B(c, \rho)$, the test reduces to $\langle q, c \rangle + \rho \|q\|_2 < \tau$ — an $O(d)$ operation independent of the number of keys inside the ball (see Section 3 of the main paper for derivation). For an AABB, the test requires two dot products ($g = 2$) but often achieves tighter pruning than a ball.

Grouping. To apply enclosing, keys must first be partitioned into groups, each enclosed by a single shape. A crucial design decision is the group size.

Definition 2 (Grouping). A grouping $\mathcal{G}(P)$ of a key set P with $|P| = n$ is a partition of P into $\lceil n/r \rceil$ disjoint subsets of size at most r , where $r = O(1)$ is a fixed group size constant.

Using a fixed group size r (rather than a fixed number of groups K) is essential. With K fixed groups and n keys, each group contains $\Theta(n/K)$ keys; as n grows, groups grow larger, their enclosing shapes become larger and more likely to intersect any query halfspace, and pruning power vanishes. With fixed r , the number of groups grows as $\Theta(n/r)$, groups remain small, and enclosing shapes stay tight — preserving pruning power as the KV cache grows. Note that prior work on KV cache clustering (e.g., for similarity search) typically uses a fixed K ; our setting requires fixed r instead.

Achieving high pruning. Given a grouping and an enclosing, the index filters groups by testing each enclosing against the query halfspace, then performs exact dot-product checks only on keys from surviving groups. The asymptotic cost per query is proportional to $g/r + f_{\text{scan}}$, where g is the gate cost per group (in dot-product equivalents) and f_{scan} is the fraction of keys not pruned. Speedup over brute-force is $1/(g/r + f_{\text{scan}})$. The empirical ablation below measures how different grouping strategies and enclosing choices affect f_{scan} and speedup on real LLM key/query data.

G.2 Empirical Ablation: Grouping Strategy and Subspace Count

Table 7 reports the fraction of keys scanned and the resulting speedup for four grouping strategies across $S \in \{2, 4, 8, 16\}$ subspaces, measured on Llama-3.2-3B-Instruct ($N \approx 4,500$, layer 15, $r=4$ keys per group). All configurations achieve perfect recall ($= 100\%$) by the halfspace enclosing guarantee. Contiguous grouping assigns consecutive key positions to the same group, which concentrates positional locality and yields the strongest pruning. At $S=16$, contiguous grouping scans only 16.3% of keys — a $2.42\times$ speedup over brute-force — while interleaved achieves a comparable 17.2% scan fraction ($2.37\times$). PCA and random strategies achieve lower pruning at the same S , because they do not exploit the local key-space structure that contiguous grouping leverages.

Table 7: Pruning ablation: scanned fraction (%) and speedup ($\times$) by grouping strategy and S . All entries: recall = 100%, $r=4$, Llama-3.2-3B-Instruct, $N \approx 4,500$.

Strategy	$S = 2$		$S = 4$		$S = 8$		$S = 16$	
	Scan%	Speed	Scan%	Speed	Scan%	Speed	Scan%	Speed
Contiguous	89.7	0.87 $\times$	72.0	1.03 $\times$	46.1	1.41 $\times$	16.3	2.42$\times$
Interleaved	90.9	0.86 $\times$	80.1	0.95 $\times$	55.0	1.25 $\times$	17.2	2.37 $\times$
Random	90.9	0.86 $\times$	82.7	0.93 $\times$	62.9	1.14 $\times$	23.8	2.05 $\times$
PCA	84.5	0.91 $\times$	73.9	1.01 $\times$	52.9	1.28 $\times$	24.1	2.04 $\times$

Table 8 reports the effect of clustering method and enclosing geometry on scanned fraction, at $r=4$ keys per group ($N \approx 8,500$, Llama-3.2-3B-Instruct, layer 14). Among clustering methods, kcenter and kmeans produce tighter groups than pq_subspace, yielding better pruning with ball and span-ball enclosings. The axis-aligned bounding box (AABB) enclosing consistently provides the most aggressive pruning (39–61% scanned fraction at $r=4$), at the cost of a higher gate evaluation cost ($g=2$ dot products per cluster vs. $g=1$ for ball and span-ball). LOUVER adopts the ball enclosing with subspace_kcenter clustering, which achieves a favorable balance between gate cost and pruning power.

Table 8: Ablation over clustering and enclosing geometry. Scanned fraction (%) and recall, $r=4$, $N \approx 8,500$, Llama-3.2-3B-Instruct, layer 14. Gate cost g : dot-product equivalents per cluster.

Clustering	Enclosing	Scan (%)	Recall	Gate g
k-center	Ball (centroid)	92.4	99.95%	1
	AABB	60.8	99.96%	2
	Span-ball	93.6	99.95%	1
k-means	Ball (centroid)	91.3	99.95%	1
	AABB	64.6	99.97%	2
	Span-ball	92.3	99.95%	1
PQ subspace	Ball (centroid)	95.7	99.81%	1
	AABB	65.7	99.83%	2
	Span-ball	89.3	99.74%	1

H How LOUVER Addresses the Design Challenges

In this section, we discuss how our index, LOUVER, addresses the design challenges introduced in Appendix F.

Challenge 1 (High Pruning Power with Full Recall). LOUVER guarantees zero false negatives by construction. For Query 1, pruning a cluster in *any* subspace requires that no point inside its bounding ball can satisfy the per-subspace threshold, which in turn implies no point can satisfy the global threshold τ (as shown in Section 4.2). For Query 2, the TA stopping condition $U(d^*) < \tau$ certifies that every unseen key has full dot product below τ . In both cases the final exact dot-product check is exact, so the returned set $\mathcal{K}^*$ is precisely $\{k \in \mathcal{K} \mid \langle q, k \rangle \geq \tau\}$ with no omissions.

Pruning power comes from two complementary sources. First, subspace decomposition amplifies pruning: a cluster must survive the ball test in *all* S subspaces to remain a candidate, so pruning in any single subspace eliminates the cluster entirely. Crucially, each ball test operates in the reduced d_s -dimensional subspace rather than the full d -dimensional space. In lower dimensions, a ball of fixed radius subtends a smaller solid angle relative to an arbitrary halfspace, so the probability of intersection is lower and more clusters are pruned per subspace. Second, balanced PCA tree grouping ensures that clusters are compact — the dominant variance axis aligns nearby keys — so bounding balls are tight and the ball-halfspace test rejects many clusters cheaply. Together these properties yield strong pruning on real LLM key distributions, as confirmed empirically in Appendix G.

Challenge 2 (Dynamic and Query-Adaptive). LOUVER handles the dynamic nature of decoding through its buffer-and-update mechanism (Section 4.3). New keys are appended to buffer $\mathcal{B}$ and attended densely with no filtering, ensuring correctness from the moment a key is generated. Once the buffer fills, the same index construction procedure runs on the buffered keys, and the resulting groups are appended directly to the main index without any rebuild.

Query-adaptivity is inherent to the range searching formulation: both query algorithms accept a threshold value τ at runtime, with no assumptions on the number of relevant keys. The retrieved set $\mathcal{K}^*$ adapts automatically to the query and the current context, handling the oscillating score distributions observed in Figure 1c without any modification to the index.

Challenge 3 (Efficiency and Integration). LOUVER is designed for hardware efficiency at every level. Fixed group size r enables fully regular memory access patterns during both the ball filter and the exact check, avoiding conditional branching and enabling vectorized execution. On GPU, the index updates run concurrently with attention, hiding their cost behind the main computation. The ball filter operates only on the $K = n/r$ group centers, which is a factor of r smaller than the full key array; the full keys are accessed only for the surviving candidate set, directly reducing memory bandwidth pressure. On CPU, the amortized update cost is $O(S \cdot d_s/r)$ per step — constant in n — while query cost over the index is sublinear in n for typical LLM score distributions.

All query and update operations are backed by hardware-optimized CUDA and CPU kernels. As shown in Section 5, LOUVER *outperforms even highly optimized dense attention implementations such as FlashAttention [12]*.

I Pseudo-codes

Algorithm 1 LOUVER Index Construction

Input: Keys $\mathcal{K} = \{k_1, \dots, k_n\} \subset \mathbb{R}^d$, subspace count S , group size r
Output: Index $\mathcal{I} = (\{c_{s,i}, \rho_{s,i}, a_{s,i}\}_{s=1}^S)$

- 1: Compute slices $(\delta_0, \dots, \delta_S)$ with $\delta_s - \delta_{s-1} \in \{\lfloor d/S \rfloor, \lceil d/S \rceil\}$
- 2: **for** $s = 1$ **to** S **do**
- 3: $k_j^{(s)} \leftarrow \pi_s(k_j)$ for all j ▷ project onto subspace s
- 4: $\{a_{s,j}\} \leftarrow \text{BALANCEDPCATREE}(\{k_j^{(s)}\}, r)$ ▷ group assignments
- 5: **for** $i = 1$ **to** K **do**
- 6: $c_{s,i} \leftarrow \frac{1}{r} \sum_{j: a_{s,j}=i} k_j^{(s)}$ ▷ group center
- 7: $\rho_{s,i} \leftarrow \max_{j: a_{s,j}=i} \|k_j^{(s)} - c_{s,i}\|_2$ ▷ ball radius
- 8: **return** $\mathcal{I}$

Algorithm 2 BALANCEDPCATREE

Input: Points $P = \{p_1, \dots, p_m\} \subset \mathbb{R}^{d_s}$, group size r , next cluster id *offset* (default 0), assignment array $a[\cdot]$ (shared, modified in place)
Output: $a[j]$ set for all $j \in \{1, \dots, m\}$

- 1: **if** $m \leq r$ **then** ▷ leaf: assign all points to one group
- 2: $a[j] \leftarrow \text{offset}$ for all $j \in \{1, \dots, m\}$
- 3: **return**
- 4: $\ell \leftarrow \lfloor m/2 \rfloor$ ▷ left subtree size
- 5: $\hat{e} \leftarrow \arg \max_{e \in \{e_1, \dots, e_{d_s}\}} \text{Var}_{p \in P}(\langle p, e \rangle)$ ▷ dominant coordinate axis
- 6: Sort P by $\langle p, \hat{e} \rangle$; let $L \leftarrow P_{1:\ell}$, $R \leftarrow P_{\ell+1:m}$
- 7: $\text{BALANCEDPCATREE}(L, r, \text{offset}, a)$
- 8: $\text{BALANCEDPCATREE}(R, r, \text{offset} + \lceil \ell/r \rceil, a)$

Algorithm 3 LOUVER Query 1 — Full-Subspace Ball Filter

Input: Query $q \in \mathbb{R}^d$, per-subspace thresholds $(\tau_s)_{s=1}^S$, global threshold τ , index $\mathcal{I}$, buffer $\mathcal{B}$, scale σ
Output: Attention output $o \in \mathbb{R}^d$

- 1: **for** $s = 1$ **to** S **do** ▷ ball filter per subspace
- 2: $\mathcal{L}_s \leftarrow \bigcup \{ \mathcal{C}_{s,i} : \langle \pi_s(q), c_{s,i} \rangle + \rho_{s,i} \|\pi_s(q)\|_2 \geq \tau_s \}$
- 3: $\mathcal{L} \leftarrow \mathcal{L}_1 \cap \mathcal{L}_2 \cap \dots \cap \mathcal{L}_S$ ▷ AND across subspaces
- 4: $\mathcal{L}^* \leftarrow \{j \in \mathcal{L} : \langle q, k_j \rangle \geq \tau\}$ ▷ exact dot-product check
- 5: $o \leftarrow \text{ATTENTION}(q, \mathcal{K}[\mathcal{L}^*] \cup \mathcal{B}, \sigma)$
- 6: **return** o

Algorithm 4 LOUVER Query 2 — TA Filter

Input: Query $q \in \mathbb{R}^d$, threshold $\tau \in \mathbb{R}$, index $\mathcal{I}$, buffer $\mathcal{B}$, scale σ

Output: Attention output $o \in \mathbb{R}^d$

```
1: for  $s = 1$  to  $S$  do
2:    $f_{s,i} \leftarrow \langle \pi_s(q), c_{s,i} \rangle + \rho_{s,i} \|\pi_s(q)\|_2$    for all  $i$ 
3:    $\sigma_s \leftarrow \text{argsort}_i(-f_{s,i})$                                  $\triangleright$  descending upper bounds
4:  $\mathcal{L} \leftarrow \emptyset, d \leftarrow 0$ 
5: repeat
6:    $d \leftarrow d + 1$ 
7:   for  $s = 1$  to  $S$  do
8:      $\mathcal{L} \leftarrow \mathcal{L} \cup \mathcal{C}_{s, \sigma_s(d)}$                  $\triangleright$  add all members of scanned cluster
9:      $U \leftarrow \sum_{s=1}^S f_{s, \sigma_s(d)}$                  $\triangleright$  upper bound on unseen keys
10: until  $U < \tau$ 
11:  $\mathcal{L}^* \leftarrow \{j \in \mathcal{L} : \langle q, k_j \rangle \geq \tau\}$          $\triangleright$  exact dot-product check
12:  $o \leftarrow \text{ATTENTION}(q, \mathcal{K}[\mathcal{L}^*] \cup \mathcal{B}, \sigma)$ 
13: return  $o$ 
```

J Update Process

J.1 GPU Update

On GPU, the incremental update runs concurrently with the ongoing attention computation using a two-stream design. The index maintains a pre-allocated arena with sufficient capacity for all expected update rounds; no memory reallocation occurs during decoding.

The update is split into two phases:

- **Phase 1** (side stream, concurrent with attention): the B buffered keys are clustered into B/r new groups; their centers, radii, assignments, and key/value data are written into the unused tail of the arena. New slots are marked *invalid* throughout Phase 1, so the attention kernel on the main stream treats them as empty and skips them — no locking or synchronization is required.
- **Phase 2** (publish, on attention stream): after the attention stream waits on the update-done event, the invalid flags for the new slots are flipped atomically and the live-count counters are advanced. The new groups become visible to all subsequent queries from this point on.

Because Phase 1 and attention run concurrently, the wall-clock cost of an update round is hidden behind the attention computation in the common case.

J.2 CPU Update

On CPU, the update runs synchronously after every B decoding steps. The total work per update round is $O(S \cdot B \cdot \log(B/r))$ for the balanced PCA tree builds across all S subspaces, plus $O(S \cdot B \cdot d_s)$ for computing centers and radii. Amortized over the B steps between triggers, the per-step cost is $O(S \cdot d_s \cdot \log(B/r))$, which is constant in n and negligible relative to the $O(n \cdot d)$ cost of dense attention.

K Kernel Implementation Details

All performance-critical operations in LOUVER are implemented as custom CUDA and CPU kernels. On GPU, the TA filter runs as a three-stage fused pipeline (score, depth, alive compaction) in a single cooperative kernel launch, followed by a sparse FlashAttention-style kernel over the surviving candidate set. On CPU, the filter and attention are implemented in C++ with AVX/FP16 intrinsics.

K.1 GPU Filter Pipeline

The TA filter is implemented as three CUDA kernels fused into a single cooperative kernel launch via `cudaLaunchCooperativeKernel`, with `cooperative_groups::grid_group::sync()` barriers

separating phases. This avoids separate kernel dispatch overhead and keeps intermediate data in shared memory across phases.

Score kernel (Phase 1). For each query head h_q and subspace s , this kernel computes $f_{s,i} = \langle \pi_s(q), c_{s,i} \rangle + \rho_{s,i} \|\pi_s(q)\|_2$ for all K clusters and retains the top- L scores. Each thread maintains a min-heap of size $L/\text{IPT_L}$ over its assigned clusters; a CUB BlockRadixSort then produces the globally sorted top- L per (h_q, s) . This avoids sorting all K clusters — only $L \ll K$ entries enter the sort.

GQA reuse. The grid is launched as $(H_{kv}, 4)$ rather than $(H_q, 4)$. Each block streams the K centers for one (h_{kv}, s) pair from global memory *once* and scores all G query heads that share this KV head simultaneously. This reduces center GMEM traffic by a factor of G (approximately $3\times$ for Llama, $7\times$ for Qwen).

Depth kernel (Phase 2). One block per query head scans the sorted top- L scores to find the TA stopping depth d^* (smallest d with $U(d) < \tau$) and writes a per-subspace bitmask `parent_alive_bitmap` $[H_q, 4, \text{K_words}]$ marking which clusters survive. This also resets the live-count counter used by Phase 3.

Alive compaction kernel (Phase 3). Grid: $(H_q, \text{N_TILES})$ with $\text{TILE_N} = 2048$ keys per tile. Each thread loads packed cluster assignments using 128-bit vectorized loads (4 assignments per `ld.b128`, folding the invalid-key sentinel into the packing). The bitmask is read once from global memory into shared memory per tile; per-key lookups are then shared-memory reads. Alive keys within each warp are compacted via `__ballot_sync`; a CUB BlockScan computes prefix offsets; each lane writes its alive key directly to the output `live_idx` array in global memory. One `atomicAdd` per block (not per warp) claims a contiguous output range.

K.2 GPU Sparse Attention Kernel

The sparse attention kernel receives the compact `live_idx` list from the filter and computes scaled dot-product attention restricted to the surviving keys. The sequence is split into P splits; each split computes partial softmax statistics (m_p, ℓ_p, o_p) in the FlashAttention [12] reduction scheme. A final combine pass merges the P partials into the output o . Buffer keys $\mathcal{B}$ are appended to each split to ensure they are always attended densely.

K.3 CPU Kernels

The CPU filter is implemented in C++ with AVX2/FP16 intrinsics. Ball scores are computed with vectorized dot products over the d_s -dimensional center vectors; the AND-across-subspaces pruning is done with 64-bit bitmask operations. The CPU attention kernel performs a fused dot-product + softmax over the surviving candidates, with inner loops fully unrolled at compile time using the fixed group size r as a template parameter. Both kernels operate entirely within L2/L3 cache for typical sequence lengths, since only the $K = n/r$ centers need to be streamed during filtering.

L Discussion

Integration with inference engines. LOUVER is designed to integrate with existing LLM inference systems with minimal changes. vLLM [26] manages the KV cache in fixed-size pages; by reordering children within each parent group into contiguous memory, child fetches from GPU parents remain cache-friendly and hardware-efficient — directly compatible with vLLM’s block allocator. We also provide a drop-in cache and attention module compatible with HuggingFace Transformers and PyTorch, enabling use with any LLM architecture without modification.

Broader impact. This work identifies false negatives in sparse attention as a first-class source of accuracy degradation, a direction not well-studied in prior work. We hope the halfspace range-searching framing opens a principled path for future work on sparse attention correctness, beyond heuristic top- k or eviction strategies.

Limitations. LOUVER’s primary design objective is retrieval correctness and low latency; reducing GPU memory usage is a secondary goal. The index introduces parent cluster centers as additional GPU-resident data: a memory overhead that is small relative to the full KV cache (28% at 1 GB; see Table 4), but non-zero.

One approach to eliminate this overhead entirely is to replace synthetic cluster centers with actual keys: select the key closest to each group's centroid as the parent representative and store only a reference (index) to it rather than a new d -dimensional vector. This is analogous to how hierarchical ANN indexes (e.g., HNSW [35]) use existing data points as navigating nodes, avoiding any additional storage.